U-Net 3D : positions 3D et segmentation d'instance de bactéries

Holographie inline, hologrammes 1024 × 1024 (pixel 0,175 µm), profondeur jusqu'à 207 µm. Répertoire `deep_learning_segmentation/` — état du 2026-10-22.

Ce que fait la chaîne

Deux réseaux U-Net 3D de même architecture travaillent l'un après l'autre sur le champ reconstruit de l'hologramme :

- **Étage 1 — positions.** Sur tout le volume reconstruit à basse résolution (0,35 × 0,35 × 1 µm), il produit une carte des centres des bactéries ; ses maxima au-dessus d'un seuil donnent les positions (x, y, z).
- **Étage 2 — segmentation d'instance.** Autour de chaque position, le champ est refocalisé exactement sur une fenêtre fine (0,175 × 0,175 × 0,25 µm, 8 × 7 × 7 µm) ; le réseau y dessine le masque de la bactérie et repère les centres des voisines. Un second tour segmente les bactéries collées que l'étage 1 avait fusionnées.
- **Sortie.** Pour chaque bactérie : son masque 3D, son barycentre, son axe, sa longueur, son épaisseur et son volume ; export CSV (colonnes d'HOLOTRACKER) et instances .npz.

Les deux réseaux sont entraînés sur des hologrammes **100 % simulés, sans fond** (fond plat + bruit capteur), avec le masque d'instance écrit par le simulateur lui-même. Les jeux sont produits par `GUI_generate_dataset.py` (BMP, volumes d'instance, positions CSV, `parameters.yaml`).

Chiffres clés (modèles retenus, models/stage1 et models/stage2)

	U-Net	Méthode classique
Détection, test simulé 200 µm, N = 50–400 (F1)	0,938	0,826
Film réel : bactéries suivies par HOLOTRACKER retrouvées*	93 % (100 détections / image)	100 % (a produit la vérité ; 192 maxima / image)
Paires côte à côte séparées à 1,5 µm / 2 µm	98 % / 100 %	0 % / 0 %
Paires l'une au-dessus de l'autre séparées à 4 µm	58 %	90 %
Précision de position, étage 1 (σx / σz)	0,11 / 0,31 µm ; affinée par le barycentre du masque : σz 0,21 µm	0,22 / 0,55 µm
Segmentation : bactéries d'IoU > 0,5 ; erreur d'axe	53 % ; 11° (médiane)	—
Temps par hologramme (200 µm, fp16)	1,23 s + 0,08 à 0,54 s (étage 2)	2,6 s (400 plans)

* Vérité partielle : les ≈ 49 bactéries par image reliées en trajectoire par HOLOTRACKER (seuil MAD local probablement, voir section 7). Le rappel se compare entre modèles à nombre de détections égal ; la précision n'est pas mesurable ainsi. Limites mesurées : l'IoU voxel à voxel plafonne vers 0,53 même sur les vraies positions (la forme fine est sous la résolution optique, 2 µm en XY et 5 µm en Z) ; la taille (± 10 %) n'est pas mesurable ; deux bactéries l'une au-dessus de l'autre ne sont séparées qu'à partir d'environ 6 µm. Certaines paires et bactéries rapprochées des jeux s'interpénètrent (47 % des paires à 1 µm entre centres).

1. Structure du répertoire

Élément	Rôle
`models/stage1/`	étage 1 retenu (échelle fixe, entraîné le 2026-10-18 sur l'ancien jeu seg_sim) : model_best.pt (poids + configuration + seuil), config.json, history.json, train_summary.json
`models/stage2/`	étage 2 retenu (mêmes fichiers)
`holo_physics.py`	reconstruction par spectre angulaire, décimation ×2 par le spectre ; normalisations (MAD / écart-type de l'hologramme ou du film, MAD locale) ; cibles de l'étage 1
`unet3d.py`	U-Net 3D résiduel, repli de la BatchNorm à l'inférence
`losses.py`	loss focale CenterNet (retenue), MSE pondérée, BCE + Dice, focale + Tversky
`detection.py`	inférence par tuiles, pics, appariement hongrois, méthode classique de référence
`seg_stage2.py`	étage 2 : refocalisation locale, cibles, loss, inférence en deux tours, dédoublonnage, mesures
`GUI_generate_dataset.py, simu_dataset.py`	génération d'un jeu simulé (BMP + instances .npz + positions CSV + parameters.yaml) et sa lecture par l'entraînement
`generate_dataset.py`	ancien format .npz (jeu seg_sim des modèles retenus, paires rapprochées des benchmarks)
`generate_sequences.py`	films 100 % simulés en mouvement (dynamique du film réel)
`train.py, train_stage2.py`	entraînement des deux étages (lisent les deux formats)
`benchmark.py, benchmark_seg.py, plot_seg_examples.py`	évaluation : détection (avec balayage du seuil), chaîne complète, exemples en trois coupes → resultats/
`inference_pipeline.py`	inférence sur un film, sans interface ; export CSV + .npz
`GUI_inference.py, GUI_train.py, gui_messages.py`	interfaces ; calcul hors de la boucle Tk, messages par paquets bornés, arrêt propre (fichier STOP)
`datasets/`	jeux de GUI_generate_dataset.py : dataset_sim_100um_N20-400, dataset_sim_200um_N20-400, dataset_sim_100um_N10-200 (≈ 950 Mo ; à ne pas versionner)
`runs/`	entraînements de l'étude des normalisations (11 modèles d'étage 1, non versionné)
`resultats/`	benchmarks et journaux (non versionné) ; benchmark_grid_B/RAPPORT_NORMALISATION_ETAGE1.pdf
`experiences/`	scripts des expériences du rapport (variantes de l'étage 1, fonds hybrides)
`RAPPORT_UNET3D.md, README.md, PIPELINE_UNET3D.pdf`	rapport, mode d'emploi, ce document
`HELP_TRAIN.pdf, HELP_INFERENCE.pdf`	aide détaillée : chaque champ des GUI et chaque option des scripts
`doc_tools/`	génération des trois PDF à partir du code (schémas, tableaux des réglages)

Le dossier data/ a été supprimé (ancien jeu seg_sim des modèles retenus, films data/seq) ; les deux se régénèrent : gen_seg_sim_data.sh (generate_dataset.py, graines 401 à 404) et generate_sequences.py.

2. Pipeline d'inférence

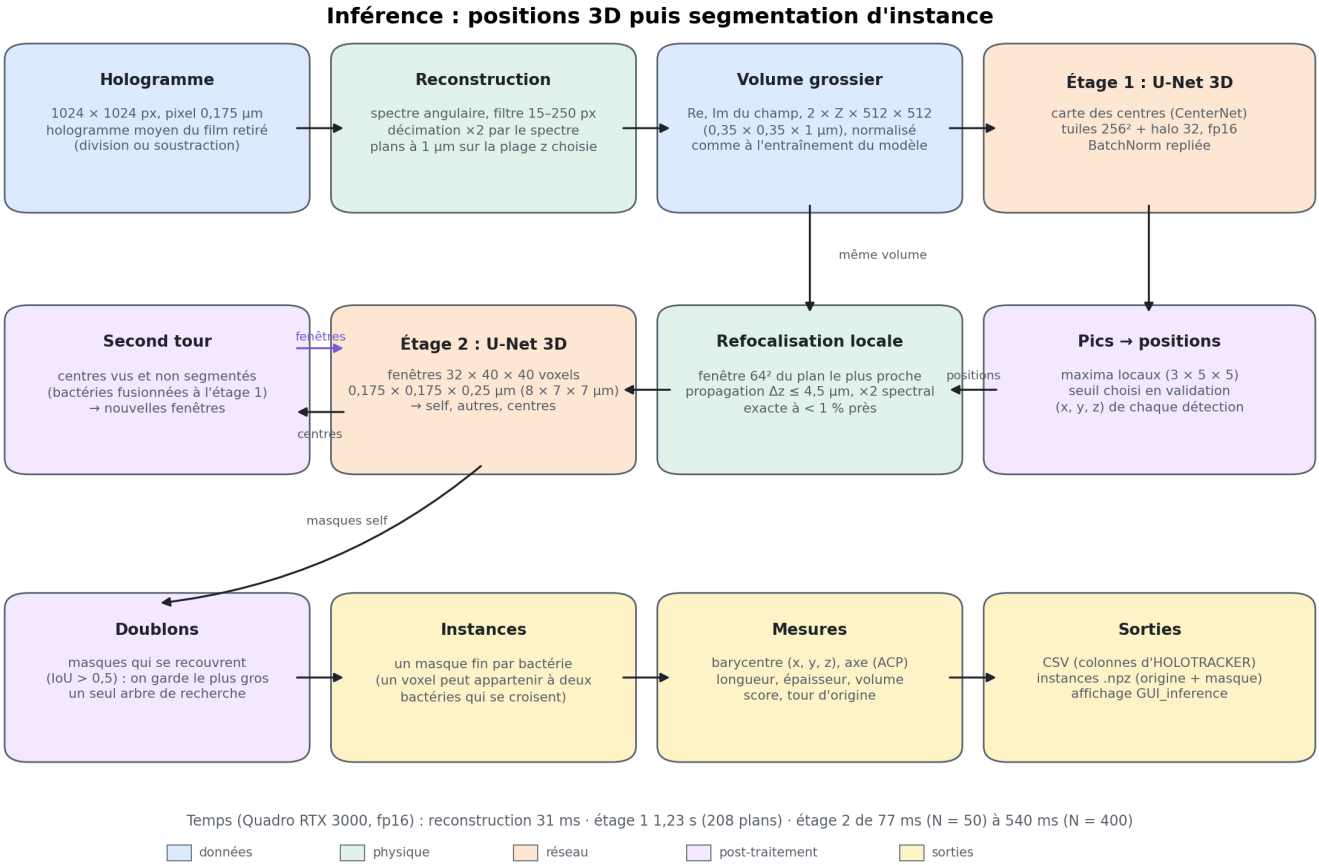


Figure 1 — de l'hologramme aux instances. Le volume grossier, calculé une seule fois, sert aux deux étages.

- **Suppression de l'hologramme moyen.** L'hologramme moyen du film est retiré par division (I / M, comme HOLOTRACKER : retire aussi un éclairage non uniforme) ou par soustraction ((I − M) / ⟨M⟩ + 1). Sans film : division par la moyenne de l'image.
- **Reconstruction.** Filtre 15–250 px de Fourier, propagation par spectre angulaire au pas de 1 µm sur la plage z choisie (seuls les plans utiles sont calculés, 8 µm de marge). La décimation latérale ×2 recadre le spectre : toute la bande utile est conservée, le coût est divisé par 4. Plage z, pas, décimation, filtre et optique se règlent dans GUI_inference.py (onglet Reconstruction) ; vides, ce sont les valeurs d'entraînement du modèle, et tout écart est signalé.
- **Normalisation.** Re et Im sont mis à l'échelle comme à l'entraînement du modèle (section 4) ; pour une normalisation « film », les statistiques sont calculées sur 32 images du film traité.
- **Étage 1.** Le réseau lit le volume par tuiles de 256 px (+ 32 px de halo) et produit une carte de chaleur des centres ; les maxima locaux (fenêtre 3 × 5 × 5 voxels par défaut, réglable) au-dessus du seuil sont les détections. Le seuil retenu en validation simulée est enregistré avec le modèle ; il se règle à l'inférence, et c'est le réglage qui pèse le plus sur le nombre de détections d'un film réel.
- **Refocalisation locale.** Pour chaque détection, un carré (64 px par défaut) du plan grossier le plus proche est propagé de Δz ≤ 4,5 µm vers les plans fins et suréchantillonné dans le spectre d'un facteur égal à la décimation (1, 2 ou 4). Le champ obtenu est identique à moins de 0,4 % près à une reconstruction directe à pleine résolution, pour une fraction du coût. L'étage 2 exige la décimation de son entraînement.
- **Étage 2 et second tour.** Chaque fenêtre donne le masque de « sa » bactérie et une carte fine des centres de toutes les bactéries visibles. Les centres qui ne correspondent à aucune instance déjà trouvée (bactéries collées, fusionnées par l'étage 1) deviennent de nouvelles fenêtres.
- **Doublons et mesures.** Deux instances proches (1,5 µm en XY, 3 µm en Z) dont les masques se recouvrent (IoU > 0,5) sont fusionnées. Chaque masque fournit barycentre, axe (ACP), longueur, épaisseur et volume. Seuils, fenêtres des maxima et fusion des doublons se règlent dans l'onglet Réglages de GUI_inference.py (HELP_INFERENCE.pdf).

3. Données et entraînement

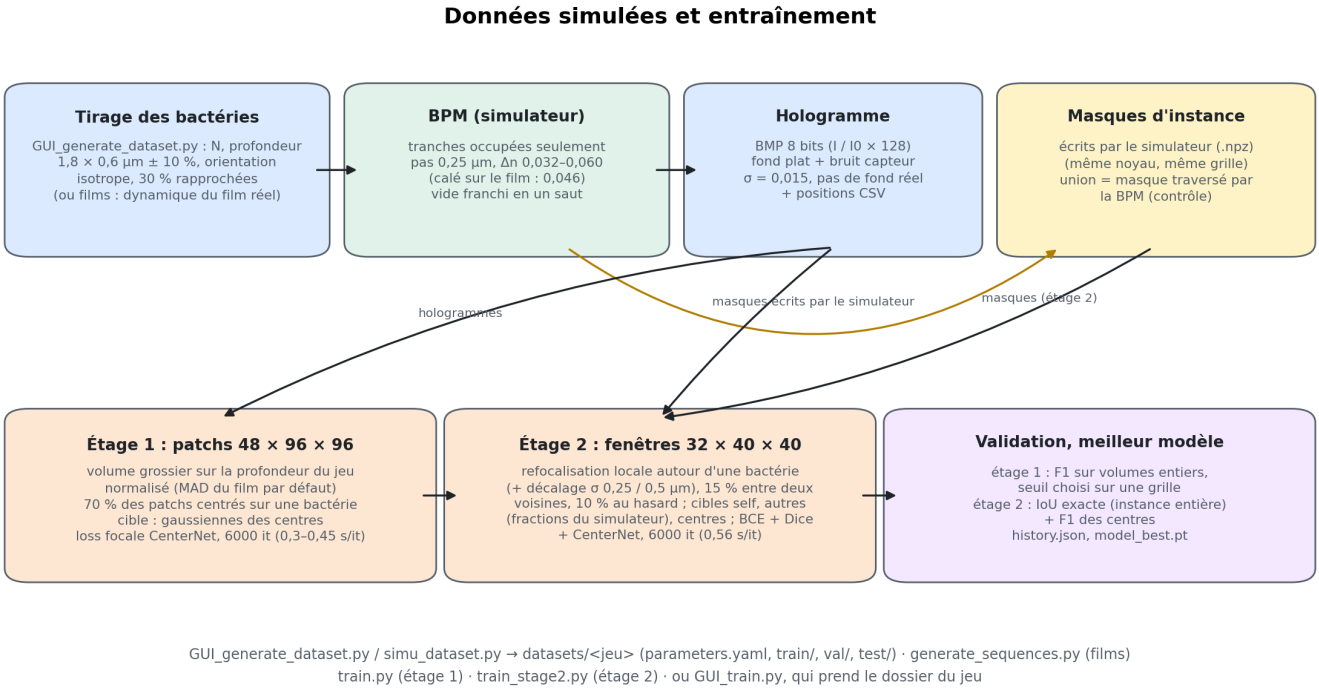


Figure 2 — génération des données simulées et entraînement des deux étages.

- **Génération.** GUI_generate_dataset.py reprend les paramètres de simulation_gui.py (volume, optique, caméra, bactéries, illumination) et écrit, pour train/, val/ et test/, les hologrammes BMP 8 bits (gris = I / I0 × 128), les volumes d'instance (.npz creux), les positions (CSV, z = distance au capteur) et un parameters.yaml avec tous les paramètres. La génération tourne dans un processus séparé, arrêt propre possible.
- **Simulation.** Bâtonnets de 1,8 × 0,6 µm (± 10 %), orientations isotropes, 30 % placés à 0,6–4 µm d'un autre pour apprendre les contacts. BPM au pas de 0,25 µm : seules les tranches occupées sont traversées pas à pas, le vide est franchi en une propagation (identique à 10^{-5} près à une propagation plan par plan sur tout le volume). Δn tiré entre 0,032 et 0,060 autour de la valeur calée sur le film réel (0,046). Fond plat + bruit capteur σ = 0,015.
- **Masques.** Le simulateur écrit le masque de chaque bactérie avec le même noyau et la même grille que la BPM ; à la génération, l'union de ces masques est comparée au masque réellement traversé.
- **Étage 1.** GUI_train.py prend le dossier du jeu ; le volume reconstruit couvre automatiquement la profondeur du jeu (distance volume-caméra → distance + plans × pas). Patchs de 48 × 96 × 96 voxels tirés à 70 % autour d'une bactérie, symétries exactes, cible gaussienne des centres, loss focale CenterNet ; normalisation MAD du film par défaut ; 6000 itérations (≈ 30 à 45 min).
- **Étage 2.** Fenêtres refocalisées autour des bactéries (décalées comme le serait une détection), 15 % entre deux voisines (détection fusionnée), 10 % au hasard (fausse détection). Cibles : la bactérie de la fenêtre et les autres (fractions du masque du simulateur), centres ; BCE + Dice + CenterNet ; grille fine au pas des masques du jeu ; 6000 itérations (≈ 55 min).
- **Films en mouvement.** generate_sequences.py simule la dynamique mesurée sur le film réel (40 % de nageuses à 0,86 µm/image, 60 % diffusives) et conserve l'identité de chaque bactérie.
- **Réglages.** Tous les paramètres de l'entraînement (reconstruction : décimation, filtre, pas Z ; cibles : largeur des gaussiennes ; fenêtres : patchs, fenêtre fine, carré refocalisé ; attribution, fenêtre des maxima…) sont des constantes en tête de train.py et train_stage2.py, des options de la ligne de commande et des champs de GUI_train.py ; les valeurs utilisées sont enregistrées avec le modèle et reprises par l'inférence. Détail : HELP_TRAIN.pdf.

4. Normalisation de l'entrée

Avant d'entrer dans l'étage 1, le volume reconstruit (Re, Im) est centré et divisé par une échelle σ, la même pour Re et Im (la phase est conservée). Le choix est enregistré avec le modèle et repris à l'inférence.

Normalisation (train.py --norm)	Centre et échelle	Calculés sur
filmmad — MAD du film (défaut de GUI_train.py)	médiane ; 1,4826 × MAD	32 hologrammes du film (médiane des valeurs)
filmstd — écart-type du film	moyenne ; écart-type	32 hologrammes du film
globalmad — MAD de l'hologramme	médiane ; 1,4826 × MAD	le volume de chaque hologramme (+ 65 ms)
globalstd — écart-type de l'hologramme	moyenne ; écart-type	le volume de chaque hologramme
localmad — MAD locale	médiane ; MAD par maille, interpolées	mailles de 13 × 11,2 µm (réglables)
fixed — échelle fixe (modèle retenu)	0 ; 0,015 × √(part du spectre filtré)	rien : constante du bruit capteur

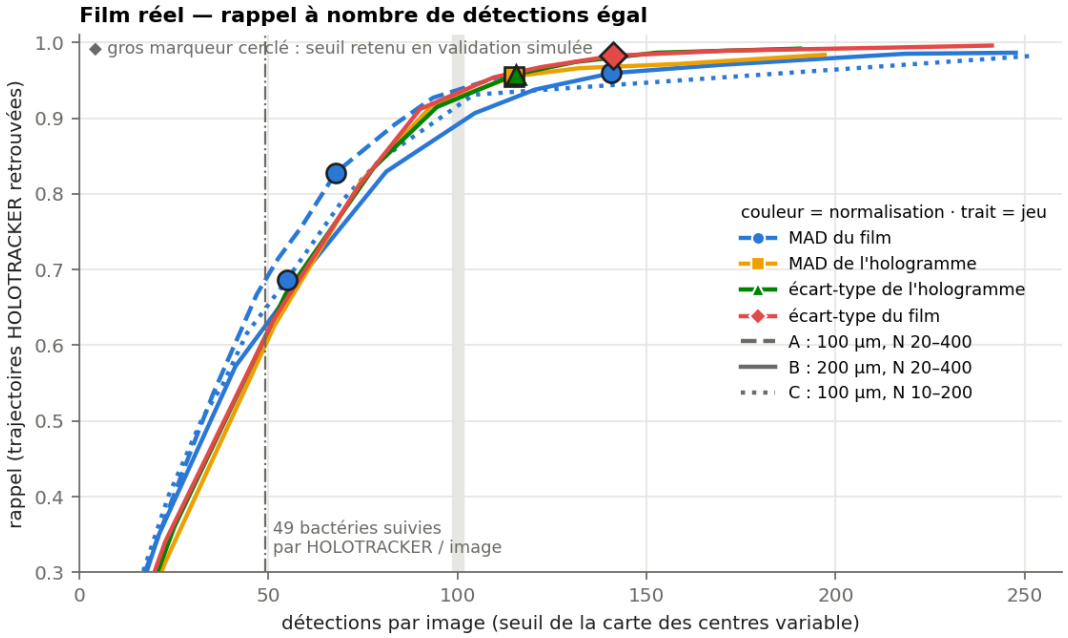


Figure 3 — film réel : rappel en fonction du nombre de détections par image (seuil variable), six modèles entraînés sur trois jeux (100 µm ou 200 µm, densité ×1 ou ×½) avec quatre normalisations mesurées.

- **À nombre de détections égal, aucune normalisation ni aucun jeu ne se détache** : rappel de 0,89 à 0,94 pour 100 détections par image, 0,93 pour le modèle retenu à échelle fixe. Les grands écarts vus au seuil de validation venaient du nombre de détections que ce seuil donne sur le film réel (de 55 à 141 par image selon le modèle).
- **Le fond du volume suit la densité** : écart-type et MAD du volume sont multipliés par 2,1–2,2 entre 100 et 400 bactéries (franges des bactéries hors focus). Une normalisation par hologramme affaiblit donc une même bactérie dans un hologramme dense ; une normalisation par film reste constante d'une image à l'autre et s'adapte au bruit d'un autre montage, d'où le défaut « MAD du film ».
- Une seule graine par variante, un seul film réel : détails dans resultats/benchmark_grid_B/RAPPORT_NORMALISATION_ETAGE1.pdf.

5. Architecture du réseau

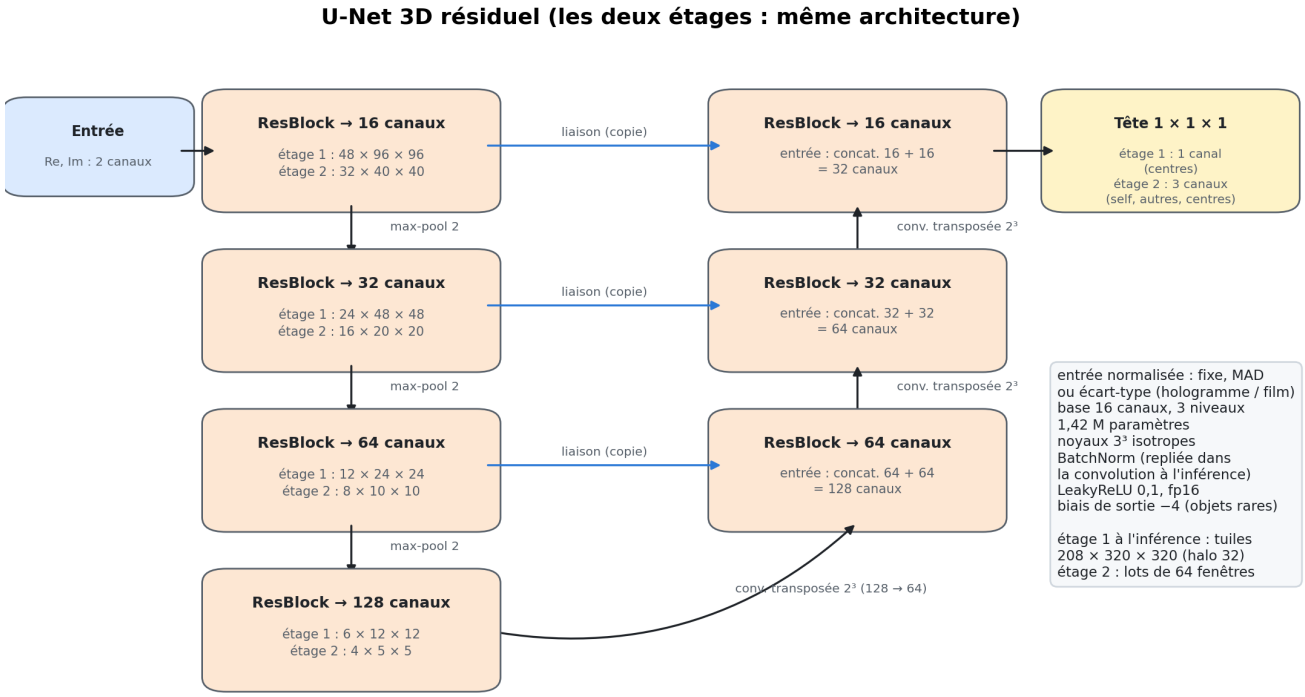


Figure 4 — U-Net 3D résiduel ; tailles des cartes pour un patch de l'étage 1 et une fenêtre de l'étage 2.

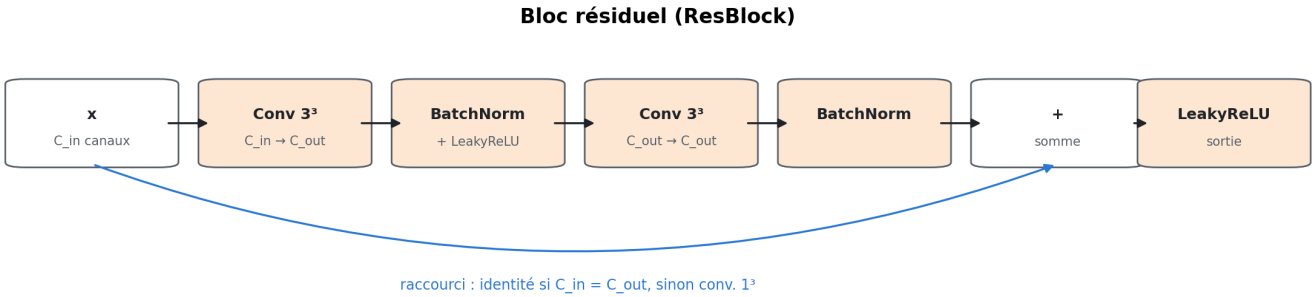
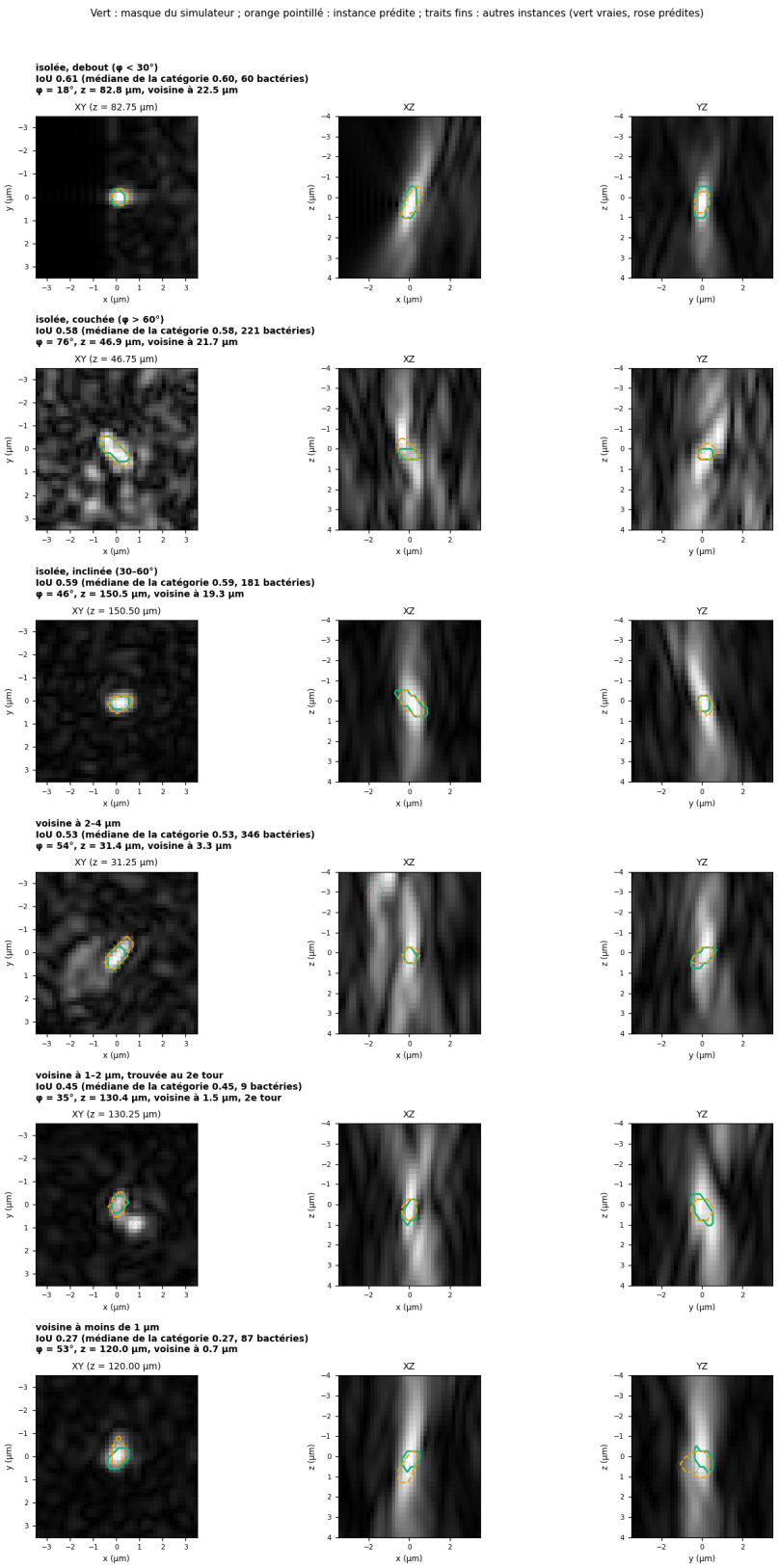


Figure 5 — bloc résiduel utilisé à chaque niveau, à la descente comme à la remontée.

Choix	Valeur	Raison
Entrée	2 canaux : Re, Im du champ, normalisés (section 4)	la bactérie, transparente, est un objet de phase : le champ complexe garde toute l'information
Profondeur, canaux	3 sous-échantillonnages ; 16 → 32 → 64 → 128	champ récepteur de plusieurs dizaines de voxels, bien plus large que la réponse d'une bactérie (≈ 6 voxels) : il voit les voisines
Noyaux	3³ isotropes (voxel 0,35 × 0,35 × 1 µm)	la réponse fait environ 6 voxels dans chaque direction
Couches de normalisation	BatchNorm, repliée dans la convolution à l'inférence	GroupNorm mesurée 2,4× plus lente
Sortie	1 canal (étage 1) ; 3 canaux (étage 2)	carte des centres ; self, autres, centres
Paramètres	1,42 M (5,7 Mo)	inférence en fp16

6. Segmentation : exemples en trois coupes

Bactéries typiques du test simulé (IoU la plus proche de la médiane de leur catégorie). Fond : amplitude du champ refocalisé ; vert : masque du simulateur ; orange pointillé : instance prédite.



7. Résultats des modèles retenus

Modèles du 2026-10-18 (étage 1 à échelle fixe), évalués sur l'ancien jeu seg_sim (200 µm, N = 50–400).

Détection (étage 1), test simulé 200 µm

F1 selon N	50	100	200	300	400	Rappel	Précision	Couchées
U-Net, étage 1	0,945	0,951	0,941	0,939	0,931	0,890	0,992	0,858
Classique (400 plans)	0,803	0,798	0,816	0,837	0,834	0,777	0,881	0,738

Film réel (images 85–99, vérité : 49 trajectoires HOLOTRACKER par image)

Détections par image de l'étage 1	Nombre
correspondent à une trajectoire (rappel 0,934)	46,0
au-delà de 100 µm (hors plage d'HOLOTRACKER)	14,6
sur un objet non suivi de la taille d'une bactérie	19,8
doublons (< 2 µm d'une détection juste)	6,3
inexpliquées	13,4

Vérité : TRAJECTORIES_20261016_173113.csv (points reliés en trajectoire, ≈ 49 des ≈ 84 détections d'HOLOTRACKER par image). Les réglages d'HOLOTRACKER ne sont pas dans ses journaux ; le fichier last_param.json, écrit entre les deux passes, indique seuil MAD local K = 25, focus SUM_OF_VARIANCE, 200 plans de 0,5 µm (0–100 µm) — probablement ceux utilisés.

Segmentation (chaîne complète)

	Rappel	Précision	F1	PQ	IoU > 0,5	σxy / σz
Étage 1 seul	0,890	0,992	0,938	—	—	0,15 / 0,31 µm
Chaîne, 1 tour	0,889	0,993	0,938	0,361	0,525	0,12 / 0,21 µm
Chaîne, 2 tours	0,911	0,990	0,949	0,360	0,531	0,12 / 0,21 µm
Sur les vraies positions	0,980	0,997	0,989	0,376	0,574	0,10 / 0,16 µm

Voisine la plus proche	< 1 µm	1–2 µm	2–4 µm	4–8 µm	> 8 µm
Bactéries segmentées (IoU > 0,5)	0,09	0,35	0,57	0,58	0,65

Temps (1024 × 1024, fp16)

	Reconstruction	Réseau / focus	Post-traitement	Total
Étage 1, 200 µm	31 ms	1377 ms	39 ms	1447 ms
Étage 2, N = 50 / 400	—	—	—	77 / 542 ms
Classique, 200 µm (400 plans)	195 ms	510 ms	1906 ms	2611 ms
Étage 1, 100 µm, pas Z 2 µm	12 ms	261 ms	22 ms	294 ms

8. Utilisation

Interfaces

- `python deep_learning_segmentation/GUI_generate_dataset.py` — paramètres de simulation_gui.py + nombre d'hologrammes par jeu, plage du nombre de bactéries, plage d'indice ; écrit datasets/<jeu> (parameters.yaml, train/, val/, test/) ; aperçu du dernier hologramme, arrêt propre.
- `python deep_learning_segmentation/GUI_train.py` — onglet Données : dossier du jeu (résumé de parameters.yaml) et profondeur reconstruite (vide : celle du jeu) ; onglets Étage 1 (normalisation, MAD du film par défaut ; maille de la MAD locale) et Étage 2 ; chaque tâche tourne dans un processus séparé, courbes de loss et de validation en direct ; « Arrêter » = arrêt propre.
- `python deep_learning_segmentation/GUI_inference.py` — film (BMP, TIFF, PNG), suppression de l'hologramme moyen (division ou soustraction), modèles (models/stage1, models/stage2 par défaut), onglet Reconstruction (pixel caméra, grossissement, λ, indice, z min / max, pas Z, décimation, filtre), seuil de l'étage 1 ; « Image n° » calcule une image, « Lancer le lot » traite une plage et exporte ; un clic sur une bactérie ouvre ses coupes XY, XZ, YZ.

Ligne de commande

Commande	Effet
`python deep_learning_segmentation/simu_dataset.py <dossier avec parameters.yaml>`	génère un jeu sans interface
`python deep_learning_segmentation/train.py --name s1 --norm filmmad --train_dir <jeu>/train --val_dir <jeu>/val`	entraîne l'étage 1 (→ runs/s1) ; --z_min_um / --z_max_um : profondeur (défaut : celle du jeu)
`python deep_learning_segmentation/train_stage2.py --name s2 --data <jeu>`	entraîne l'étage 2 (→ runs/s2)
`python deep_learning_segmentation/benchmark.py --runs models/stage1 --datasets dataset_sim_200um_N20-400`	benchmark de détection, film réel et balayage du seuil compris
`python deep_learning_segmentation/benchmark_seg.py --data dataset_sim_200um_N20-400`	benchmark de la chaîne complète (--pairs_dir : paires rapprochées)
`python deep_learning_segmentation/inference_pipeline.py <film> <sortie> --mean_removal division --z_max 100`	inférence sans interface (aussi --z_min, --z_step, --decim, --high_pass, --low_pass)
`python deep_learning_segmentation/generate_sequences.py --out <dossier>`	films simulés en mouvement

Pour utiliser un nouveau modèle, copier son dossier de runs/ dans models/ (ou le choisir dans le GUI). Lancer depuis la racine du dépôt ; sur cette machine, définir PYTHONIOENCODING=utf-8.

Chaque champ des GUI et chaque option des scripts : HELP_TRAIN.pdf (génération et entraînement) et HELP_INFERENCE.pdf (inférence). Détails, protocoles et expériences : RAPPORT_UNET3D.md ; normalisations : resultats/benchmark_grid_B/RAPPORT_NORMALISATION_ETAGE1.pdf.